

Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática

Trabajo Procesos de Ingeniería Software
Grado en Ingeniería Informática
Ingeniería del Software

La Grada Colchonera: base SaaS y proceso de desarrollo

Javier Vela González-Román

Octubre, 2026

TRABAJO DE PROCESOS DE

INGENIERÍA DEL SOFTWARE

Grado en Ingeniería Informática

Ingeniería del Software

La Grada Colchonera: base SaaS y proceso de desarrollo

Autor: Javier Vela González-Román

Octubre, 2026

Declaración de autoría

Yo, Javier Vela González-Román , declaro que soy el autor del trabajo titulado “La Grada Colchonera: base SaaS y proceso de desarrollo”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores, detallándose asimismo el uso responsable de la inteligencia artificial en el Anexo C.

Albacete, Octubre, 2026

Fdo.: Javier Vela González-Román

Resumen

Esta memoria describe el diseño, la implementación y el proceso de desarrollo del Sprint 1 del proyecto La Grada Colchonera, una aplicación SaaS organizada en capas con gestión de usuarios, desarrollada como parte de la asignatura Procesos de Ingeniería del Software. El trabajo abarca desde la estructuración inicial de un servidor Express con Node.js y una base de datos relacional en PostgreSQL hasta el establecimiento de un flujo de trabajo riguroso basado en Scrum mediante DoneTonic, control de versiones con GitHub Flow, integración continua y despliegue automático en la nube a través de Google Cloud Run. Se detallan exhaustivamente la arquitectura en capas del sistema, el modelo de datos, la gestión segura de sesiones y contraseñas mediante hash, así como la evaluación de la calidad mediante pruebas automatizadas y la revisión de requisitos funcionales y no funcionales.

Palabras clave: SaaS, Scrum, GitHub Flow, Node.js, PostgreSQL, Google Cloud Run, Ingeniería del Software.

Índice general

1 Introducción, Motivación y Objetivos 1

1.1 Motivación 1

1.2 Objetivos 1

1.3 Alcance de la entrega 2

1.4 Estructura de la memoria 3

2 Propuesta de Solución, Metodología y Herramientas 5

2.1 Propuesta de solución 5

2.2 Metodología 5

2.2.1 Gestión con Scrum en DoneTonic 6

2.2.2 Flujo de versiones: GitHub Flow adaptado 6

2.2.3 Pipeline de CI/CD 7

2.3 Herramientas 9

3 Descripción de la Aplicación 11

3.1 Requisitos del sistema 11

3.2 Arquitectura de la solución 11

3.2.1 Modelo de datos 11

3.2.2 Ciclo de sesión y control de acceso 14

3.2.3 Especificación de la API 14

3.3 Decisiones técnicas y de diseño 14
3.4 Desarrollo del proyecto 16
3.5 Manual de usuario: La Grada Colchonera 16
3.5.1 Página de inicio y registro 16
3.5.2 Inicio de sesión y gestión de cuenta 16
3.5.3 Panel de administración y control de acceso 16
4 Evaluación 23
4.1 Diseño de la evaluación 23
4.2 Resultados 23
4.3 Discusión 24
5 Conclusiones y Trabajo Futuro 25
5.1 Conclusiones 25
5.2 Trabajo futuro 26
A Desarrollo del Proyecto 27
A.1 Sprint 1 27
A.1.1 Planificación del Sprint 1 27
A.1.2 Desarrollo del sprint 30
A.1.3 Revisión del sprint 30
A.1.4 Retrospectiva 32
B Material Adicional 35
B.1 Enlaces del proyecto 35
B.2 Credenciales de administrador para la revisión 35
C Uso Responsable de la Inteligencia Artificial 37
C.1 Uso de la inteligencia artificial en el desarrollo del software 37
C.2 Uso de la inteligencia artificial en la redacción de la memoria 38
Referencia bibliográfica 39

Índice de figuras

2.1 Tablero del Sprint 1 en DoneTonic: cada fila es un PBI con sus tareas por columnas de estado; los PBIs en verde están hechos. 7
2.2 Pipeline CI/CD: push/PR -> CI -> si main y verde -> Cloud Run + Render. . . . 8

3.1 Arquitectura en capas y flujo de interacción del sistema. 13
3.2 Pantalla de inicio de la aplicación con comprobación del estado del servidor. 17
3.3 Formulario de registro con mensaje de error de validación. 17
3.4 Notificación de error ante credenciales incorrectas en el inicio de sesión. . 18
3.5 Vista de Mi cuenta con datos de usuario y opción de baja de servicio. . . . 18
3.6 Redirección y denegación de acceso ante un intento de navegación no autorizada. 19
3.7 Panel de administración con listado y gestión de usuarios. 20
3.8 Panel de administración en móvil: cada usuario como tarjeta. 21

A.1 Burndown del Sprint 1: evolución de PBIs pendientes del alcance de la entrega. 33
A.2 Sprint Burnup y Sprint Burndown en DoneTonic (84% completado, 3 PBIs pendientes). 33

Índice de tablas

2.1 Clasificación de herramientas empleadas en el proyecto. 9

3.1 Requisitos funcionales y no funcionales del Sprint 1. 12
3.2 Endpoints expuestos por la API REST. 15
3.3 Resumen del Sprint 1 del proyecto. 16

A.1 Sprint Backlog del Sprint 1 (19 PBIs, 16 hechos en el alcance). 28
A.2 Trazabilidad tarea → PR de los PBIs hechos (P1–P14, P18 y P19; 26 tareas). 29
A.3 Problemas y soluciones en desarrollo y pruebas (guía §8). 30
A.4 Problemas y soluciones en despliegue Google Cloud (guía §8). 31
A.5 Problemas y soluciones en Git y GitHub (guía §8). 31
A.6 Problemas y soluciones en alcance y revisión (guía §8). 31
A.7 Problemas y soluciones en memoria LaTeX (guía §8). 32
A.8 Problemas y soluciones en DoneTonic (guía §8). 32

1. Introducción, Motivación y Objetivos

1.1. Motivación

El desarrollo de software en entornos modernos trasciende la mera codificación de funcionalidades: exige la articulación de procesos de ingeniería rigurosos y reproducibles que conecten de manera fluida el espacio de trabajo local con el entorno productivo. En este marco formativo se encuadra la asignatura Procesos de Ingeniería del Software, correspondiente al curso 2026-2027 del Grado en Ingeniería Informática. El propósito pedagógico de la asignatura no radica únicamente en construir un producto funcional, sino de forma prioritaria en evaluar exhaustivamente el proceso de ingeniería aplicado [Universidad de Castilla-La Mancha, 2026]: desde la descomposición sistemática de requisitos y la trazabilidad de tareas, hasta el flujo de integración continua, el control de versiones y el despliegue automático en la nube.

La aplicación que articula este proyecto se denomina *La Grada Colchonera*. Se concibe como una plataforma social tipo red social orientada a la afición colchonera, donde compartir opiniones e interactuar en torno a la actualidad deportiva. Es fundamental subrayar que se trata de un proyecto de aficionado sin relación oficial con el Club Atlético de Madrid, por lo que carece de escudos, logotipos o cualquier elemento gráfico institucional bajo propiedad intelectual del club deportivo.

En esta primera fase correspondiente al Sprint 1, el foco de la aplicación radica en establecer una base sólida de software como servicio (*Software as a Service*, SaaS). Dicha base implementa la gestión integral de cuentas de usuario sobre una arquitectura organizada en capas desacopladas, asegurando los pilares de seguridad, persistencia y mantenibilidad antes de acometer en entregas posteriores las funcionalidades sociales complejas.

1.2. Objetivos

El objetivo principal del proyecto consiste en diseñar, implementar y desplegar la estructura base de una aplicación SaaS modular con gestión de usuarios, sustentada en un ciclo

de vida guiado por la metodología ágil Scrum y un pipeline automatizado de integración y entrega continuas sin intervención manual.

Para alcanzar el objetivo principal, se establecen los siguientes objetivos específicos:

1. **Diseño arquitectural desacoplado:** Organizar el servidor en capas independientes de API, lógica de negocio y persistencia, garantizando que el backend sirve los recursos estáticos del frontend desde el mismo origen para evitar problemas de compartición de recursos de orígenes cruzados.
2. **Gestión segura de identidades y sesiones:** Proporcionar un sistema de autenticación local con derivación criptográfica de contraseñas mediante funciones de hash seguro y gestión de sesiones persistentes en base de datos que sobrevivan a reinicios del servicio.
3. **Control de acceso basado en roles:** Establecer un esquema de autorización con al menos dos niveles (usuario estándar y administrador), permitiendo la baja lógica de cuentas y garantizando la protección de rutas críticas estrictamente en el lado del servidor.
4. **Persistencia relacional y gestión de configuración:** Integrar un sistema gestor de bases de datos relacional mediante migraciones de esquema versionadas y desacoplar por completo los secretos y variables de entorno del repositorio de código.
5. **Garantía de calidad mediante pruebas:** Elaborar un conjunto exhaustivo de pruebas automatizadas que cubran casos nominales y de error de la capa de lógica, ejecutables mediante un único comando en el entorno de desarrollo y en integración continua.
6. **Automatización del ciclo de entrega (CI/CD):** Diseñar e implantar flujos de trabajo en la nube que compilen contenedores, validen las pruebas ante una base de datos real y despliegen automáticamente en producción tras cada integración exitosa en la rama principal.
7. **Adopción de procesos ágiles y trazabilidad:** Gestionar el desarrollo mediante un backlog estructurado en DoneTonic y aplicar la disciplina de GitHub Flow con ramas dedicadas por tarea y revisiones en pull requests.

1.3. Alcance de la entrega

El calendario docente del Sprint 1 abarca del 29/09/2026 al 27/10/2026. Aunque la especificación inicial del enunciado contemplaba cuatro hitos semanales, la directriz docente para esta primera entrega limitó formalmente el alcance a los Hitos 1, 2 y 3 [Universidad de Castilla-La Mancha, 2026]. De este modo, los requisitos vinculados al Hito 4 —principalmente el inicio de sesión federado mediante proveedores externos OAuth, la confirmación de registros por correo electrónico transaccional y el refinamiento avanzado del control de errores— quedan pospuestos para la siguiente entrega del proyecto.

1.4. Estructura de la memoria

La presente memoria se organiza según la estructura recomendada por la Escuela Superior de Ingeniería Informática de Albacete:

- El **Capítulo 2** expone la propuesta conceptual de la solución sin entrar en detalles tecnológicos, detalla la metodología de trabajo adoptada (Scrum y GitHub Flow) y clasifica las herramientas empleadas.
- El **Capítulo 3** aborda la especificación de requisitos funcionales y no funcionales, describe la arquitectura del sistema y sus capas, analiza las decisiones de diseño y documenta la aplicación a modo de manual de uso guiado con capturas de pantalla.
- El **Capítulo 4** presenta el diseño y ejecución de la evaluación técnica, los resultados cuantitativos de cobertura y verificación, así como una discusión honesta sobre las garantías y limitaciones del proceso.
- El **Capítulo 5** contrasta el grado de cumplimiento de los objetivos planteados y traza las líneas principales de trabajo futuro.
- Los anexos complementan el documento: el **Anexo A** desglosa la planificación, incidencias y evolución métrica del Sprint 1; el **Anexo B** recopila los enlaces y repositorios del proyecto; y el **Anexo C** detalla el uso responsable y transparente de asistentes de inteligencia artificial.

2. Propuesta de Solución, Metodología y Herramientas

2.1. Propuesta de solución

La propuesta de solución responde a la necesidad de construir una base sólida para una plataforma SaaS comunitaria con capacidad de ampliación futura. Antes de abordar las particularidades de implementación o la elección concreta de librerías, la solución se concibe como un sistema distribuido estructurado bajo el principio de separación de responsabilidades en tres capas fundamentales: una capa de interfaz de usuario con navegación ligera, una capa intermedia que encapsula las reglas de negocio y políticas de autorización, y una capa de persistencia encargada de conservar el estado del sistema.

En el plano funcional, la solución propone un mecanismo de autenticación en el que los usuarios pueden registrarse y acceder al servicio de forma segura, garantizando la preservación de la sesión durante su navegación mediante identificadores almacenados en el servidor. Asimismo, se plantea un modelo de autorización diferenciado que distinga a los usuarios estándar de aquellos con privilegios administrativos. Los administradores disponen de visibilidad global sobre las cuentas existentes y ostentan la capacidad de revocar accesos, mientras que los usuarios conservan el control para solicitar su propia baja del sistema. Para asegurar una experiencia coherente, la solución postula que ninguna decisión de seguridad ni comprobación de permisos dependa exclusivamente de la interfaz del cliente: cualquier petición debe ser rigurosamente validada en el servidor antes de modificar el estado persistente.

2.2. Metodología

Para articular el desarrollo y garantizar la transparencia del progreso, se adoptó el marco de trabajo ágil Scrum [Schwaber and Sutherland, 2020], adaptado a las condiciones de un proyecto individual de ingeniería del software.

2.2.1. Gestión con Scrum en DoneTonic

La planificación y seguimiento del proyecto se centralizaron en la plataforma DoneTonic, dentro del espacio de trabajo *TFG* y el proyecto *Colchoneros*. El ciclo de trabajo corresponde al denominado **Sprint 1**, planificado entre el 29/09/2026 y el 27/10/2026.

A partir del análisis detallado del enunciado de la asignatura [Universidad de Castilla-La Mancha, 2026], se elaboró un *Product Backlog* compuesto por 19 elementos de la lista de producto (PBIs, numerados de P1 a P19), desglosados en un total de 32 tareas técnicas (T1 a T32). Puesto que todos los requisitos recogidos en el enunciado poseen carácter prescriptivo para la evaluación del software, la totalidad de los PBIs se catalogó con prioridad obligatoria (*MUST*).

Para velar por la integridad técnica del producto se definió una estricta Definición de Hecho (*Definition of Done*, DoD). Se consideró que una tarea técnica estaba completada exclusivamente cuando satisfacía los siguientes cuatro criterios concurrentes:

1. El código fuente reside en su rama de trabajo específica y se vincula formalmente a un pull request que referencia la tarea.
2. Todas las pruebas automatizadas superan satisfactoriamente la validación en el entorno de integración continua (verificación del check *Pruebas*).
3. El pull request correspondiente ha sido revisado y fusionado en la rama de integración del hito.
4. La documentación de soporte (como guías de despliegue o ficheros de configuración de ejemplo) ha sido actualizada coherentemente si la tarea lo requería.

Por su parte, un hito del sprint se considera completado cuando su rama integradora se ha fusionado en la rama principal, el software se encuentra desplegado de forma operativa en la nube y se ha emitido la correspondiente etiqueta de versión (*release*).

La Figura 2.1 muestra el tablero del Sprint 1 en DoneTonic al cierre de la entrega, con los PBIs de los Hitos 1–3, P18 y P19 en verde (hechos) y los tres PBIs del Hito 4 pendientes para la siguiente entrega.

2.2.2. Flujo de versiones: GitHub Flow adaptado

El desarrollo colaborativo y la gestión del código fuente siguieron las directrices de GitHub Flow [GitHub, sfb], estructuradas de forma jerárquica para reflejar los hitos pedagógicos:

- **Rama principal (`main`):** Representa en todo momento el código estable y listo para producción. Se encuentra protegida en GitHub, imposibilitando modificaciones directas y exigiendo obligatoriamente la resolución favorable del pipeline de integración continua.
- **Ramas de integración de hito (`hito-N/integracion`):** Actúan como puntos de convergencia para el conjunto de tareas pertenecientes a un mismo bloque temático semanal antes de su incorporación definitiva a la rama principal.

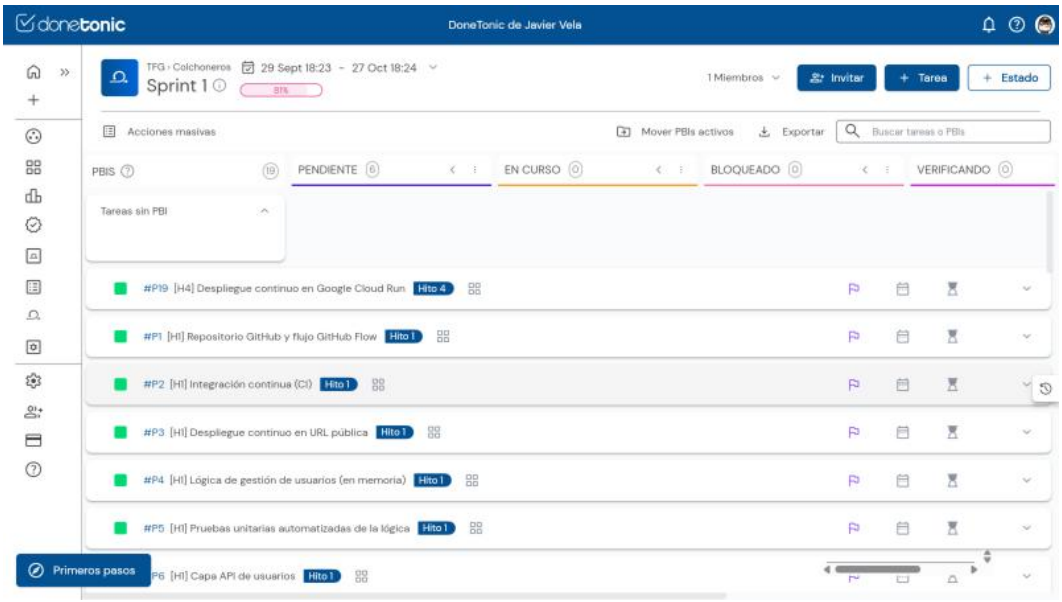


Figura 2.1: Tablero del Sprint 1 en DoneTonic: cada fila es un PBI con sus tareas por columnas de estado; los PBIs en verde están hechos.

- **Ramas de tarea (`hito-N/Tx-descripcion`):** Creadas a partir de la rama de integración correspondiente para acometer de forma aislada una única tarea del backlog.

El ciclo operativo de cada cambio se rigió por las normas estipuladas en la guía de contribución del repositorio:

1. Selección de la tarea `Tx` en el tablero de DoneTonic y creación de la rama técnica asociada.
2. Generación de commits con mensajes descriptivos en tiempo imperativo y prefijo identificativo (por ejemplo, `[Tx] Descripción del cambio`).
3. Ejecución local de la batería de pruebas unitarias y de integración (`npm test`) previo a la publicación de cambios.
4. Apertura de un pull request hacia la rama de integración del hito y verificación del paso exitoso del CI en GitHub Actions.
5. Fusión del pull request y transición automática de la tarea a estado *HECHO* en el tablero ágil.

2.2.3. Pipeline de CI/CD

El pipeline de integración y despliegue continuo, implementado con GitHub Actions, se describe en la Figura 2.2. El flujo ejecuta las pruebas contra PostgreSQL 17 real, construye la imagen Docker y, solo en `main` con CI en verde, despliega automáticamente en Google Cloud Run mediante Workload Identity Federation.

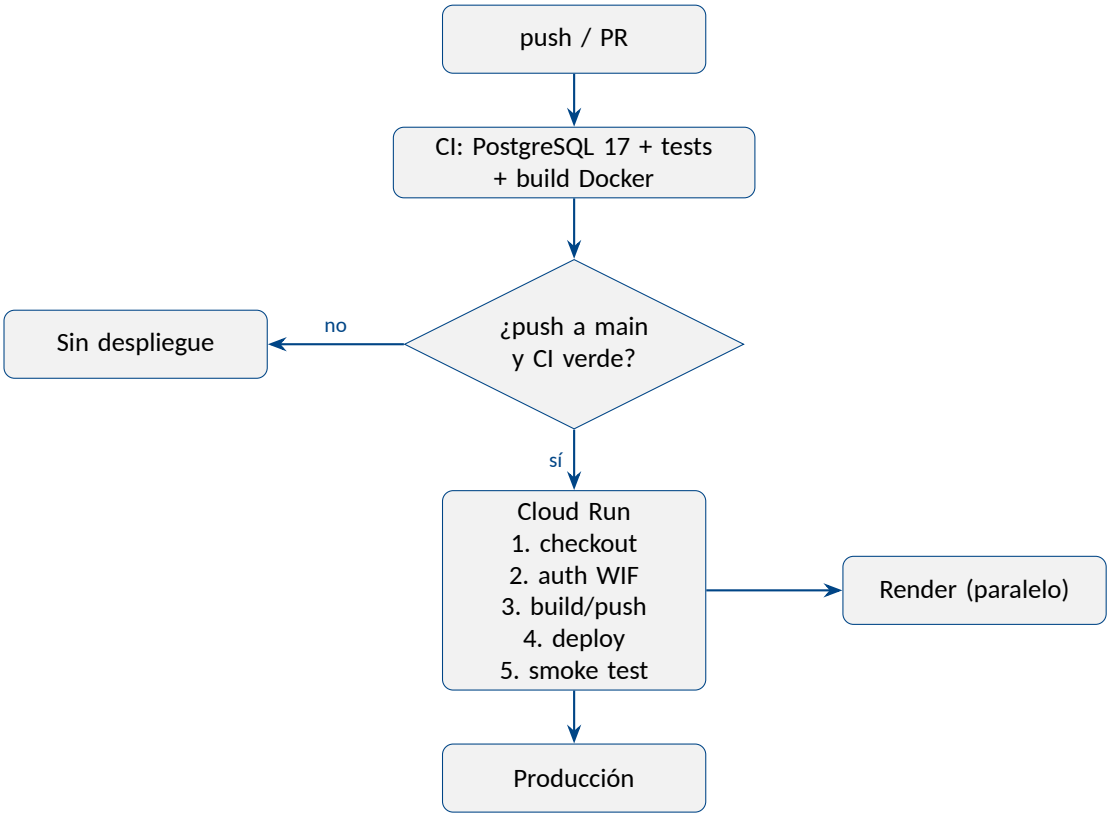


Figura 2.2: Pipeline CI/CD: push/PR -> CI -> si main y verde -> Cloud Run + Render.

2.3. Herramientas

Para sustentar las distintas dimensiones del proyecto se seleccionó un conjunto de herramientas y tecnologías de acuerdo con los requisitos técnicos exigidos en la asignatura. La Tabla 2.1 sintetiza las herramientas empleadas, estructuradas según su ámbito de aplicación en el ciclo de vida del software.

Categoría	Herramienta	Propósito en el proyecto
Gestión ágil	DoneTonic	Backlog, sprint, tablero de tareas y gráficas [Schwaber and Sutherland, 2020].
	GitHub	Repositorio público, ramas protegidas, pull requests, releases y GitHub Actions [GitHub, sfb].
Desarrollo	Node.js 22	Entorno de ejecución para unificar JavaScript en cliente y servidor [OpenJS Foundation, sf].
	Express 5	Framework web minimalista con gestión nativa de errores asíncronos [Express, sf].
	PostgreSQL (Neon)	Base de datos relacional serverless con capa gratuita en la nube [The PostgreSQL Global Development Group, sf, Neon, sf].
	Playwright	Automatización de navegador para generar capturas de interfaz homogéneas.
Despliegue y CI/CD	GitHub Actions	Pipeline de pruebas automáticas, empaquetado Docker y entrega [GitHub, sfa].
	Google Cloud Run	Plataforma PaaS de contenedores serverless para el despliegue principal [Google Cloud, sf].
	Render	Plataforma PaaS alternativa configurada con `render.yaml`.
Documentación	LuaLaTeX	Motor tipográfico para la maquetación de la memoria técnica.
	Claude (Cowork)	Asistente inteligente para la generación de código, pruebas y documentación.

Tabla 2.1: Clasificación de herramientas empleadas en el proyecto.

La combinación de estas herramientas permitió orquestar un entorno homogéneo y plenamente automatizado, reduciendo al mínimo las dependencias manuales y proporcionando trazabilidad absoluta desde la definición de requisitos hasta la verificación del despliegue en producción.

3. Descripción de la Aplicación

3.1. Requisitos del sistema

El alcance del proyecto se estructura en requisitos funcionales (RF) y no funcionales (RNF), derivados directamente de la especificación oficial de la asignatura [Universidad de Castilla-La Mancha, 2026]. La Tabla 3.1 resume estos requisitos y su estado de cumplimiento al cierre del Sprint 1.

3.2. Arquitectura de la solución

La arquitectura del sistema sigue un modelo desacoplado en tres niveles en el servidor, acompañado por la separación entre interfaz y cliente en el navegador. La Figura 3.1 ilustra la comunicación y las fronteras de responsabilidad del sistema.

Las fronteras de comunicación entre módulos están rigurosamente delimitadas:

- **Frontera GUI – Cliente:** La interfaz `public/js/ui.js` no invoca directamente funciones de red; delega toda interacción en el cliente `public/js/api.js`.
- **Frontera API – Lógica:** Los controladores de `src/api/users.js` y `src/api/auth.js` procesan peticiones HTTP e invocan los métodos de `userService`, sin contener reglas de negocio.
- **Frontera Lógica – Datos:** El servicio `userService` invoca operaciones definidas en el contrato del repositorio sin conocer si la persistencia es volátil o relacional.
- **Conexión global:** El archivo `src/app.js` es el único punto donde se inyectan las dependencias entre capas.

3.2.1. Modelo de datos

La persistencia relacional en PostgreSQL se articula mediante migraciones SQL versionadas que definen tres tablas principales:

Código	Descripción del requisito	Estado de entrega
RF-1	Backend sirve frontend desde el mismo origen sin CORS	Cumplido (Hito 1)
RF-2	Desacoplamiento estricto en tres capas (API, lógica y datos)	Cumplido (Hito 1)
RF-3	Registro de usuario local con contraseña de al menos 8 caracteres	Cumplido (Hito 2)
RF-4	Confirmación de registro por correo electrónico (estado pendiente)	Pospuesto (Hito 4)
RF-5	Autenticación local y mantenimiento de sesión persistente	Cumplido (Hito 2)
RF-6	Inicio de sesión externo mediante proveedor OAuth	Pospuesto (Hito 4)
RF-7	Cierre de sesión con destrucción en servidor e invalidación	Cumplido (Hito 2)
RF-8	Roles de usuario y administrador	Cumplido (Hito 3)
RF-9	Listado de usuarios con email, rol y estado (solo admin)	Cumplido (Hito 3)
RF-10	Comprobación de estado activo reservada al administrador	Cumplido (Hito 3)
RF-11	Eliminación de cuenta por el propio usuario o por administrador	Cumplido (Hito 3)
RF-12	Baja lógica sin reactivación y protección del último admin	Cumplido (Hito 3)
RNF-1	Persistencia relacional resistente a reinicios en PostgreSQL	Cumplido (Hito 3)
RNF-2	Secretos desacoplados en variables de entorno y Secret Manager	Cumplido (Hito 3)
RNF-3	Derivación segura de contraseñas con función de hash bcrypt	Cumplido (Hito 2)
RNF-4	Control de acceso y rutas protegidas en servidor (401 y 403)	Cumplido (Hito 2)
RNF-5	Despliegue en la nube accesible mediante URL pública	Cumplido (Hitos 1 y 4)
RNF-6	Batería de pruebas automatizadas ejecutables con un solo comando	Cumplido (Hito 1)
RNF-7	Registro de actividad en JSON estructurado por salida estándar	Cumplido (Hito 3)
RNF-8	Control de versiones con GitHub Flow e integración continua	Cumplido (Hitos 1–3)

Tabla 3.1: Requisitos funcionales y no funcionales del Sprint 1.

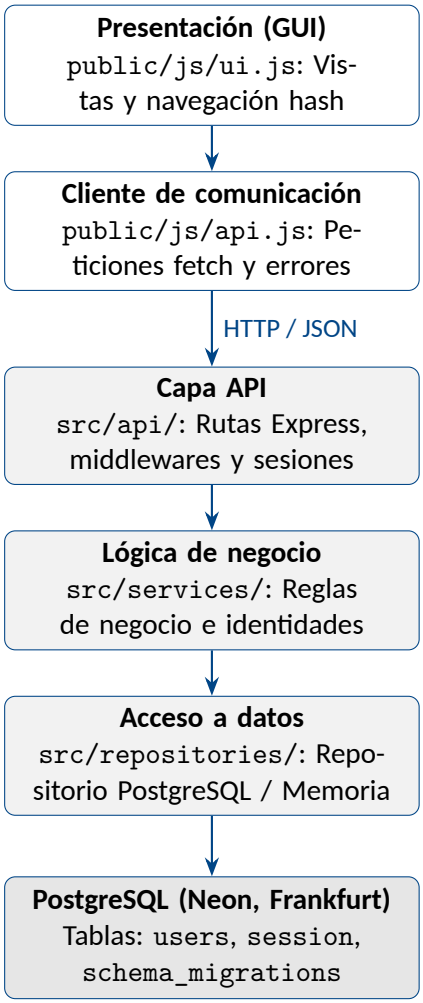


Figura 3.1: Arquitectura en capas y flujo de interacción del sistema.

- **`users`:** Almacena las identidades del sistema. Contiene los campos `id` (UUID, clave primaria), `email` (TEXT, único), `password_hash` (TEXT, hash bcrypt), `role` (`'user'` o `'admin'`), `status` (`'pending'`, `'active'` o `'deleted'`), `auth_provider` (`'local'` o `'google'`), `created_at` (TIMESTAMPTZ) y `deleted_at` (TIMESTAMPTZ).
- **`session`:** Estructurada según el estándar de `connect-pg-simple`, con campos `sid`, `sess` (JSON con datos serializados) y `expire`.
- **`schema_migrations`:** Registra los ficheros de migración ejecutados secuencialmente.

Un usuario se considera activo en el sistema cuando su atributo `status` es igual a `'active'`, lo que certifica que la cuenta no ha sido dada de baja ni permanece pendiente de confirmación.

3.2.2. Ciclo de sesión y control de acceso

El ciclo de vida de la autenticación se apoya en cookies de sesión firmadas:

1. El cliente envía credenciales mediante `POST /api/auth/login`.
2. La lógica valida la existencia del usuario y compara la contraseña mediante `bcryptjs`.
3. Ante el éxito, Express regenera la sesión para evitar ataques de fijación de sesión, asigna el identificador del usuario y guarda el registro en la base de datos relacional.
4. El servidor emite una cookie `grada.sid` con directivas `httpOnly`, `sameSite=lax` y una caducidad de 7 días.
5. Al recargar la aplicación, el cliente invoca `GET /api/auth/me`. Si el usuario hubiera sido eliminado con posterioridad, la sesión queda automáticamente invalidada.
6. Al solicitar `POST /api/auth/logout`, la sesión se destruye en la tabla `session` del servidor y se limpia la cookie del navegador.

3.2.3. Especificación de la API

La Tabla 3.2 detalla las operaciones provistas por la capa API bajo el prefijo común `/api`.

3.3. Decisiones técnicas y de diseño

Cada decisión técnica adoptada en el proyecto responde a requisitos funcionales y de seguridad:

- **Node.js 22 y Express 5:** Permite unificar JavaScript en cliente y servidor, aprovechando la captura nativa de excepciones asíncronas en Express 5 [Express, sf, OpenJS Foundation, sf].

Método	Ruta	Acceso	Propósito
GET	`/api/health`	Público	Comprobación de salud del servidor
POST	`/api/auth/register`	Público	Registro con email y contraseña (mínimo 8)
POST	`/api/auth/login`	Público	Inicio de sesión con credenciales locales
POST	`/api/auth/logout`	Público	Destrucción de la sesión en el servidor
GET	`/api/auth/me`	Sesión	Obtención del usuario con sesión activa
GET	`/api/users`	Admin	Listado de usuarios (email, rol y estado)
GET	`/api/users/:id/active`	Admin	Comprobación del estado activo
PATCH	`/api/users/:id/role`	Admin	Modificación de rol de usuario
DELETE	`/api/users/:id`	Propio / Admin	Baja lógica de cuenta de usuario

Tabla 3.2: Endpoints expuestos por la API REST.

- **Frontend sin framework compilado:** Elimina la necesidad de transpilación, asegura la entrega desde el mismo origen y delimita nítidamente la capa visual (`ui.js`) de la capa cliente (`api.js`).
- **Patrón repositorio e inyección de dependencias:** Se implementaron dos repositorios intercambiables (`memoryUserRepository` y `postgresUserRepository`) bajo la misma interfaz. Esto posibilitó migrar al almacenamiento relacional sin modificar la lógica de negocio.
- **Baja lógica (`status = 'deleted'`):** Evita la eliminación física de registros para preservar auditorías y previene que una cuenta dada de baja vuelva a autenticarse.
- **Ofuscación de mensajes de autenticación:** Se devuelve el mismo mensaje ante correos no registrados o contraseñas inválidas, evitando la enumeración de usuarios.
- **Inicialización del primer administrador (`ensureAdmin`):** Configurado al arrancar mediante `ADMIN_EMAIL` y `ADMIN_PASSWORD`. Se ejecuta de forma idempotente sin almacenar credenciales en el código fuente.
- **Protección del último administrador:** La lógica impide degradar o eliminar al último administrador, garantizando que el sistema nunca quede huérfano de gestión.
- **Logging estructurado en JSON:** Emisión por salida estándar hacia Cloud Logging, enmascarando contraseñas, secretos y tokens de sesión.

3.4. Desarrollo del proyecto

El desarrollo se organizó en torno al Sprint 1, articulando la implementación en los hitos pedagógicos semanales. La Tabla 3.3 sintetiza las dimensiones generales del ciclo, cuyo desglose detallado se analiza en el Anexo A.

Ciclo	Periodo	Objetivo central	Estado final
Sprint 1	29/09/2026 – 27/10/2026	Base SaaS con gestión de usuarios, CI/CD y despliegue cloud	16 PBIs hechos (3 pospuestos a Hito 4)

Tabla 3.3: Resumen del Sprint 1 del proyecto.

3.5. Manual de usuario: La Grada Colchonera

La interacción con *La Grada Colchonera* se realiza a través de su interfaz web de estética rojiblanca. A continuación se describe el funcionamiento de los flujos de usuario principales.

3.5.1. Página de inicio y registro

Al acceder a la URL principal, la aplicación muestra el estado operativo del servicio y los accesos públicos, tal como se observa en la Figura 3.2.

El formulario de registro valida que el correo posea un formato válido y que la contraseña cuente con una longitud de al menos 8 caracteres. Si el usuario incumple estas restricciones o introduce un correo ya registrado, el sistema bloquea el envío y muestra un mensaje explicativo, según ilustra la Figura 3.3.

3.5.2. Inicio de sesión y gestión de cuenta

Si se introducen credenciales incorrectas, la aplicación notifica el error de manera homogénea para preservar la seguridad, como se aprecia en la Figura 3.4.

Una vez autenticado, el usuario accede a la sección *Mi cuenta*, ilustrada en la Figura 3.5. En este panel se visualizan los datos del perfil y se ofrece la posibilidad de solicitar la baja definitiva de la cuenta mediante baja lógica.

3.5.3. Panel de administración y control de acceso

Si un usuario sin privilegios intenta acceder a rutas administrativas, el servidor deniega la operación con código 403 y la interfaz redirige la navegación, como se refleja en la Figura 3.6.



Figura 3.2: Pantalla de inicio de la aplicación con comprobación del estado del servidor.

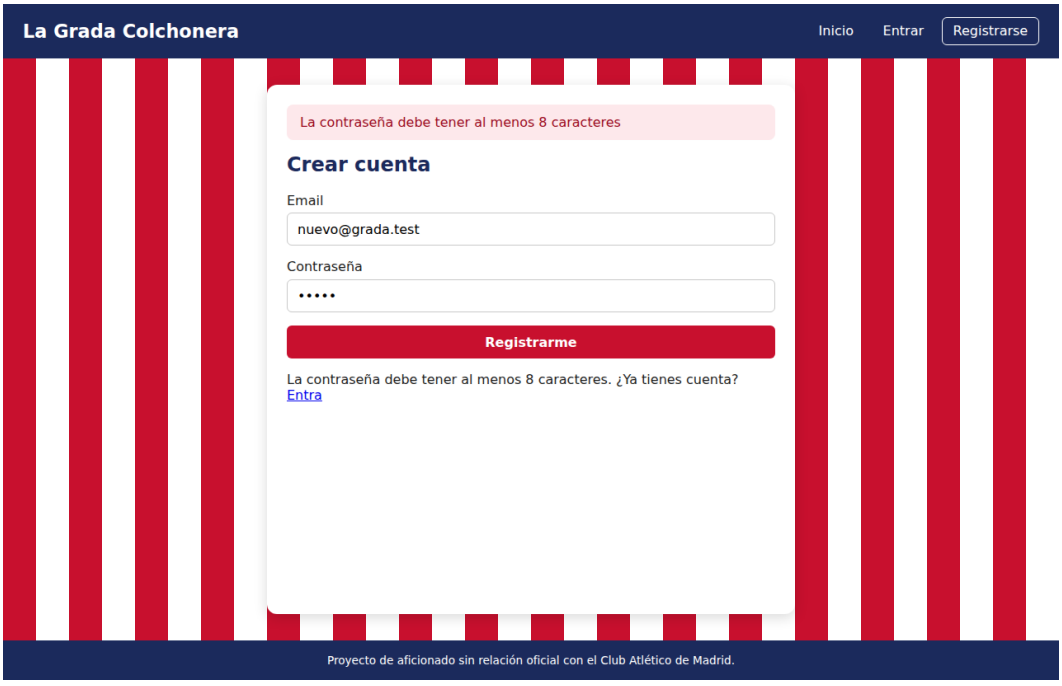


Figura 3.3: Formulario de registro con mensaje de error de validación.



Figura 3.4: Notificación de error ante credenciales incorrectas en el inicio de sesión.



Figura 3.5: Vista de Mi cuenta con datos de usuario y opción de baja de servicio.



Figura 3.6: Redirección y denegación de acceso ante un intento de navegación no autorizada.

Por el contrario, al identificarse con rol de administrador, se habilita el panel de gestión mostrado en la Figura 3.7. En esta vista se presentan todos los usuarios con sus atributos de correo, rol, estado y fecha de alta, permitiendo modificar privilegios o revocar accesos.

En dispositivos móviles, la interfaz se adapta responsivamente presentando a cada usuario como una tarjeta individualizada, garantizando la usabilidad de las acciones en pantallas estrechas (Figura 3.8).

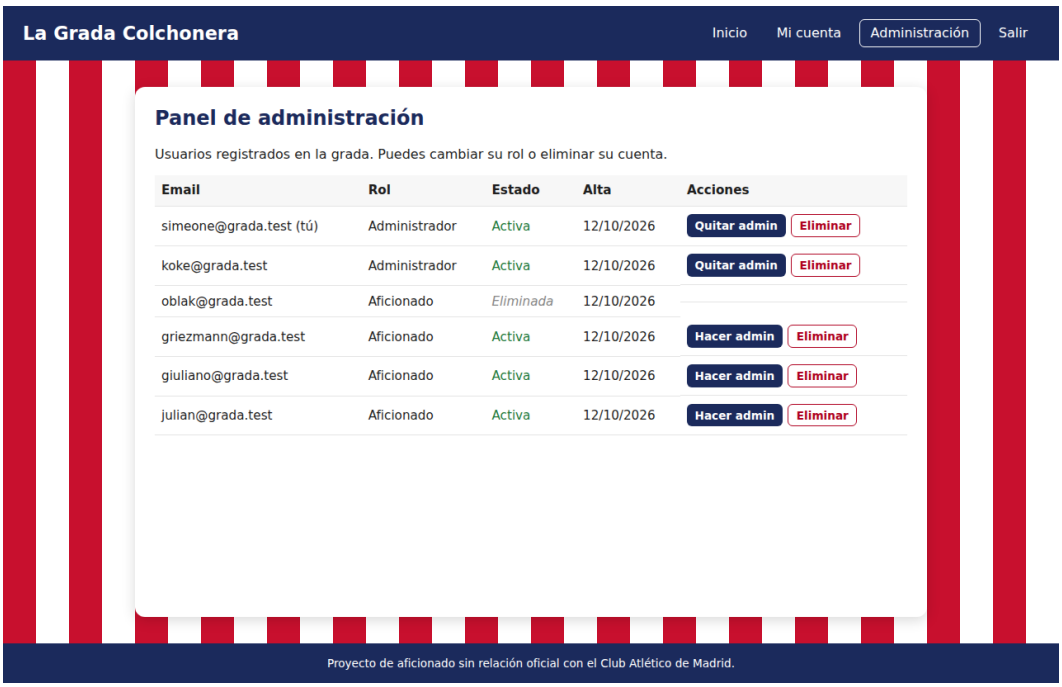


Figura 3.7: Panel de administración con listado y gestión de usuarios.

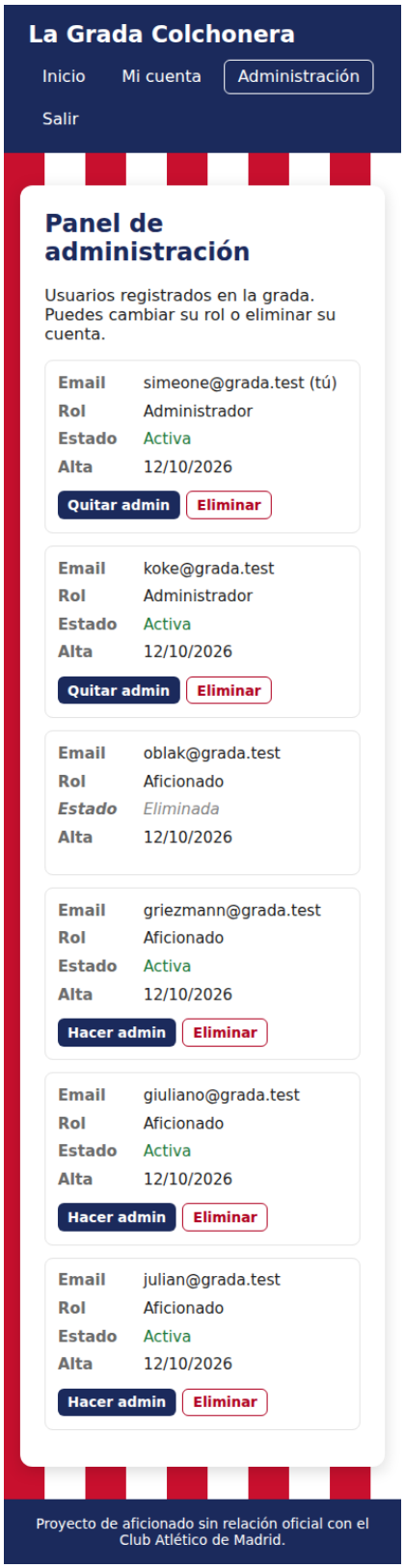


Figura 3.8: Panel de administración en móvil: cada usuario como tarjeta.

4. Evaluación

4.1. Diseño de la evaluación

La evaluación de la calidad y del cumplimiento de la especificación se diseñó como un proceso multicapa continuo y reproducible. El marco de verificación descansa en cuatro pilares metodológicos:

1. **Pruebas unitarias e integración en local:** Uso del ejecutor nativo `node:test` con el simulador de base de datos en memoria `pg-mem`, lo que permite ejecutar la suite completa en el entorno de desarrollo mediante el comando unificado `npm test`.
2. **Integración continua en GitHub Actions:** Ejecución del flujo de trabajo automatizado en cada pull request y push hacia la rama `main`. Dicho flujo despliega un servicio PostgreSQL 17 real para ejecutar las pruebas contra la base de datos auténtica y valida la construcción de la imagen de producción en Docker mediante un contenedor `node:22-alpine`.
3. **Comprobación del despliegue en Google Cloud Run:** El flujo de despliegue valida la salud del servicio desplegado mediante un smoke test que realiza hasta 20 intentos de consulta contra el endpoint `/api/health`.
4. **Revisión frase a frase contra la especificación:** Auditoría manual exhaustiva documentada en `docs/revision-entrega.md`, contrastando cada apartado del enunciado con su evidencia concreta en código y pruebas.

4.2. Resultados

La ejecución de las pruebas y la infraestructura arrojaron los resultados cuantitativos reflejados a continuación:

- **Batería de pruebas automatizadas:** Un total de 80 pruebas automatizadas (79 ejecutables en entorno local y 1 prueba de extremo a extremo que se ejecuta exclusivamente en el CI contra PostgreSQL 17). De ellas, 26 corresponden a pruebas unitarias de la

lógica de negocio en `userService`, evaluando registros duplicados, cuentas pendientes, control del último administrador y permisos. Las restantes cubren autenticación (10), API de usuarios (11), contratos de repositorios (12), logger (6), configuración (5), migraciones (3), interfaz (3), arranque (2) y salud (1).

- **Métricas del código fuente:** El servidor en `src/` cuenta con 1.128 líneas de JavaScript; el frontend en `public/` suma 657 líneas; y las pruebas en `tests/` alcanzan 992 líneas repartidas en 13 ficheros. Se crearon 2 migraciones SQL (`users` y `session`).
- **Actividad en GitHub:** Se contabilizan 65 commits en `main` (33 de trabajo y 32 fusiones de pull request), 33 pull requests integrados y 33 ramas gestionadas. Se registraron 71 ejecuciones de GitHub Actions (el workflow CI completó 65 ejecuciones).
- **Despliegue productivo:** El último commit en `main` (`978098e`, PR #32, 12/10/2026) se desplegó en Google Cloud Run en 58 s de forma automática.
- **Consumo de recursos en Neon (a fecha 12/10/2026):** 0,27 CU-h de cómputo (frente a 100 CU-h disponibles), 0,03 GB de almacenamiento (sobre un límite de 1 GB) y 0 GB de transferencia sobre un límite mensual de 5 GB [Neon, sf].

4.3. Discusión

El diseño de la evaluación proporciona garantías sólidas respecto a la integridad funcional del software: ninguna rama pudo fusionarse sin que la batería completa de pruebas resultara exitosa en un entorno PostgreSQL idéntico al de producción. Asimismo, la revisión frase a frase permitió identificar discrepancias sutiles antes del cierre, como la restricción del endpoint de verificación de usuario activo, que originalmente estaba abierto al propio usuario y fue restringido al administrador en el PR #30 para ceñirse fielmente al enunciado [Universidad de Castilla-La Mancha, 2026].

No obstante, como observación crítica sobre el proceso, debe señalarse que los 33 commits de desarrollo quedaron registrados con fecha 12/10/2026. Si bien la estructura del historial es estrictamente incremental y respeta el flujo de ramas y pull requests individuales por tarea, la concentración temporal en una única jornada debe considerarse con transparencia ante la evaluación docente, al contrastarla con la expectativa pedagógica de un avance espaciado en el tiempo.

5. Conclusiones y Trabajo Futuro

5.1. Conclusiones

El desarrollo del Sprint 1 ha permitido materializar con éxito los objetivos planteados al inicio de la memoria, combinando la construcción de una aplicación funcional con la adhesión estricta a un proceso riguroso de ingeniería del software:

- **Objetivo principal:** Se diseñó, implementó y desplegó en producción la arquitectura base de un servicio SaaS con gestión integral de usuarios, automatizando por completo el flujo desde la codificación hasta la entrega en la nube.
- **Diseño arquitectural desacoplado:** Se estructuró el backend en capas bien diferenciadas (API, lógica y datos) servidas desde el mismo origen que el cliente web, garantizando la independencia y modularidad del sistema.
- **Gestión de identidades y sesiones:** Se implementó la autenticación local con derivación de contraseñas mediante `bcryptjs` y almacenamiento de sesiones persistentes en base de datos resistente a reinicios.
- **Control de acceso basado en roles:** Se implantó la autorización diferenciada para usuarios y administradores, con baja lógica y protección de rutas en el servidor.
- **Persistencia relacional y configuración:** Se integró PostgreSQL en la nube mediante migraciones SQL versionadas y se aislaron los secretos en variables de entorno y Secret Manager.
- **Garantía de calidad mediante pruebas:** Se consolidó una suite de 80 pruebas automatizadas que cubren casos nominales y de error, ejecutables mediante un único comando.
- **Automatización del ciclo de entrega (CI/CD):** Se implantaron flujos de trabajo en GitHub Actions con validación en contenedores Docker y despliegue automático en Google Cloud Run y Render.
- **Procesos ágiles y trazabilidad:** Se gestionó el backlog en DoneTonic cumpliendo la disciplina de GitHub Flow, vinculando cada tarea a su rama y pull request.

5.2. Trabajo futuro

De cara a las siguientes fases del proyecto (particularmente el Hito 4), se identifican tres líneas prioritarias de evolución técnica:

1. **Autenticación federada y verificación por correo (P15 y P16):** Incorporar el inicio de sesión con proveedores externos OAuth (Google) y el flujo de confirmación de cuentas mediante un servicio de correo transaccional (activando `REQUIRE_EMAIL_CONFIRMATION=true`).
2. **Manejo avanzado de errores y feedback interactivo (P17):** Perfeccionar el sistema de retroalimentación al usuario en la interfaz, unificando los códigos de error y mejorando la recuperación ante fallos de conectividad.
3. **Rediseño visual y expansión funcional:** Evolucionar la estética de la interfaz hacia una experiencia comunitaria más rica una vez que se incorporen las funcionalidades de publicación e interacción social.

A. Desarrollo del Proyecto

A.1. Sprint 1

A.1.1. Planificación del Sprint 1

Objetivo y fechas

El Sprint 1 se planificó del 29/09/2026 al 27/10/2026 con el objetivo de entregar la base de una aplicación SaaS con gestión de usuarios, arquitectura en capas, proceso Scrum en DoneTonic, flujo GitHub Flow, integración y despliegue continuos (CI/CD) en Google Cloud Run y Render, y una batería de pruebas automatizadas. El alcance formal, según directriz docente, se limitó a los Hitos 1, 2 y 3 del enunciado [Universidad de Castilla-La Mancha, 2026].

Sprint Backlog

El Product Backlog inicial contenía 15 PBIs (P1–P14, P18) correspondientes los Hitos 1–3. Durante el sprint se añadió el PBI P19 (despliegue en Google Cloud Run) tras la concesión de créditos educativos de Google Cloud, elevando el total a 16 PBIs en el alcance de la entrega. La Tabla A.1 muestra el sprint backlog con sus hitos y estado final.

Trazabilidad

Cada PBI se desglosó en 1–2 tareas técnicas (T1–T32), cada una con su rama `hito-N/Tx-descripcion`, pull request `[Tx] ...` hacia `hito-N/integracion` y fusión tras CI en verde. Los cierres de hito fueron PR #9 (Hito 1), #17 (Hito 2), #24 (Hito 3), #29 (cierre sprint) y #32 (revisión final). La trazabilidad completa tarea → rama → PR figura en `docs/gestion/README.md` y en la Tabla A.2 (tareas de los PBIs hechos).

PBI	Hito	Título	Estado
P1	H1	Repositorio GitHub y flujo GitHub Flow	Hecho
P2	H1	Integración continua (CI)	Hecho
P3	H1	Despliegue continuo en URL pública	Hecho
P4	H1	Lógica de gestión de usuarios (en memoria)	Hecho
P5	H1	Pruebas unitarias automatizadas de la lógica	Hecho
P6	H1	Capa API de usuarios	Hecho
P7	H2	Frontend: interfaz y cliente de comunicación separados	Hecho
P8	H2	Registro e inicio de sesión locales	Hecho
P9	H2	Cierre de sesión y mantenimiento de la sesión	Hecho
P10	H2	Rutas protegidas en el servidor	Hecho
P11	H3	Persistencia en base de datos real	Hecho
P12	H3	Gestión de secretos con variables de entorno	Hecho
P13	H3	Roles de usuario y administrador	Hecho
P14	H3	Registro (log) de la actividad	Hecho
P15	H4	Inicio de sesión con proveedor externo (OAuth)	Siguiente entrega
P16	H4	Confirmación de cuenta por correo electrónico	Siguiente entrega
P17	H4	Manejo de errores y feedback al usuario	Siguiente entrega
P18	H4	README completo y revisión de la entrega	Hecho
P19	H4	Despliegue continuo en Google Cloud Run	Hecho

Tabla A.1: Sprint Backlog del Sprint 1 (19 PBIs, 16 hechos en el alcance).

Tabla A.2: Trazabilidad tarea → PR de los PBIs hechos (P1–P14, P18 y P19; 26 tareas).

PBI	Tarea	Descripción	PR	Estado
P1	T1	Crear repositorio y estructura base	#1	Hecho
P1	T2	Configurar GitHub Flow	#2	Hecho
P2	T3	Workflow de CI con GitHub Actions	#3	Hecho
P3	T4	Despliegue automático en proveedor cloud	#4, #10	Hecho
P4	T5	Modelo de usuario y repositorio en memoria	#5	Hecho
P4	T6	Servicio de gestión de usuarios	#6	Hecho
P5	T7	Framework de pruebas y al menos 5 pruebas	#7	Hecho
P6	T8	Endpoints de la API de usuarios	#8	Hecho
P7	T9	Cliente de comunicación con la API	#11	Hecho
P7	T10	Interfaz de usuario: vistas y navegación	#12	Hecho
P8	T11	Registro y login en el backend (con hash)	#13	Hecho
P8	T12	Formularios de registro y login en el frontend	#14	Hecho
P9	T13	Mantener la sesión al recargar y cerrar sesión	#15	Hecho
P10	T14	Middleware de autenticación y protección de rutas	#16	Hecho
P11	T15	Elegir y configurar la base de datos	#18	Hecho
P11	T16	Repositorio de BD sustituyendo al de memoria	#19	Hecho
P12	T17	Sacar los secretos a variables de entorno	#20	Hecho
P13	T18	Roles, autorización y primer administrador	#21	Hecho
P13	T19	Panel de administración y baja de cuenta	#22	Hecho
P14	T20	Configurar el logging de la actividad relevante	#23	Hecho
P18	T27	Completar el README	#25	Hecho
P18	T28	Etiquetas de hito y revisión final	#26	Hecho
P18	T31	Revisión frase a frase contra el enunciado	#30	Hecho
P18	T32	Carpeta de la memoria en LuaLaTeX	#31	Hecho
P19	T29	Contenerizar la aplicación con Docker	#27	Hecho

PBI	Tarea	Descripción	PR	Estado
P19	T30	Despliegue continuo en Cloud Run con GitHub Actions	#28	Hecho

A.1.2. Desarrollo del sprint

Los problemas técnicos surgidos y sus soluciones, extraídos de la documentación del proyecto y la guía general (§8), se resumen en las Tablas A.3, A.4, A.5, A.6, A.7 y A.8.

Problema detectado	Solución adoptada
`pg-mem` no admitía `CREATE TABLE IF NOT EXISTS` ni funciones de sesión	Consulta a `information_schema`; prueba de sesiones solo contra PostgreSQL real en CI
Cookie `secure` no guardada en local con `NODE_ENV=production` sobre HTTP	Comportamiento esperado por seguridad; capturas locales en modo desarrollo
`pkill -f "node src/server.js"` cerraba la shell invocadora	Arranque del proceso de servidor con `setsid` desde script
Tabla de administración no cabía en 640 px; botón *Eliminar* oculto	Contenedor ampliado; en móvil, cada usuario se muestra como tarjeta (Fig. 3.8)
Capturas Playwright en GitHub salían sin estilos por recursos bloqueados	Descartadas; capturas de la propia aplicación generadas correctamente
Docker Hub bloqueado en entorno en la nube	Simulación local (`npm ci --omit=dev`); validación real en job del CI

Tabla A.3: Problemas y soluciones en desarrollo y pruebas (guía §8).

A.1.3. Revisión del sprint

Al cierre del Sprint 1 (12/10/2026, commit `978098e`, PR #32) se entregaron:

- Cuatro releases publicadas en GitHub: `hito-1`, `hito-2`, `hito-3` y `sprint-1`.
- Servicio web desplegado y plenamente operativo en Google Cloud Run y Render, según se referencia en el Anexo B.
- Batería de 80 pruebas automatizadas validadas al CI contra PostgreSQL 17.
- 16 PBIs completados correspondientes al alcance de la entrega (3 PBIs pospuestos a Hito 4).

Problema detectado	Solución adoptada
Script de preparación falló en Artifact Registry por propagación de IAM	Reejecución parametrizada con `PROJECT_ID`; script idempotente
Secreto `database-url` mal introducido en Cloud Run	Creación de una nueva versión del secreto (v2) con `gcloud secrets`
URL pública de Cloud Run devolvía código 404	Primer despliegue pendiente; operativa tras finalizar el workflow
Cadena de conexión de Neon compartida en chat	Recomendación de rotar clave; aplazada en lista de tareas pendientes

Tabla A.4: Problemas y soluciones en despliegue Google Cloud (guía §8).

Problema detectado	Solución adoptada
`git push --all` rechazado en ramas de integración (*non-fast-forward*)	Sincronización mediante `git fetch` y `git branch -f`
Entorno cloud sin permisos de escritura directa en GitHub	Exportación de commits vía `git bundle`; push desde local
Bundle requería commit base ausente en repositorio local	Regeneración tomando como base el commit disponible `4b16182`
Subida de bundle con mismo nombre sobrescribía versión anterior	Asignación de nombres de fichero unívocos
Fusiones en interfaz web fallaban por comprobaciones incompletas	Espera a la validación de checks y confirmación explícita

Tabla A.5: Problemas y soluciones en Git y GitHub (guía §8).

Problema detectado	Solución adoptada
Requisitos de OAuth y confirmación por correo pertenecientes a Hito 4	Alcance limitado a Hitos 1–3 por el docente; código guardado
Endpoint `/api/users/:id/active` accesible a usuarios estándar	Restricción exclusiva al rol administrador (PR #30) tras revisión
Documentos de revisión redactados a partir de PBIs y no del PDF	Reescritura sistemática de `docs/revision-entrega.md`

Tabla A.6: Problemas y soluciones en alcance y revisión (guía §8).

Problema detectado	Solución adoptada
Error de escritura de auxiliares en subcarpetas al compilar en `build/`	Configuración de `.latexmkrc` con creación recursiva de directorios
Ausencia de librerías LuaLaTeX en entorno de pruebas	Instalación de dependencias; compilación validada en GitHub Actions

Tabla A.7: Problemas y soluciones en memoria LaTeX (guía §8).

Problema detectado	Solución adoptada
Menús apilados y clics JS ineficaces en interfaz Flutter de DoneTonic	Automatización por coordenadas comprobando árbol de accesibilidad
Renderizado distorsionado al forzar tamaño de ventana en Flutter	Capturas realizadas con resolución nativa de 800 px

Tabla A.8: Problemas y soluciones en DoneTonic (guía §8).

- 65 commits en la rama principal (33 de trabajo y 32 de integración de PRs) y 71 ejecuciones de GitHub Actions.
- Consumo en el plan gratuito de Neon a fecha 12/10/2026: 0,27 CU-h de cómputo (límite 100 CU-h), 0,03 GB de almacenamiento (límite 1 GB) y 0 GB de transferencia sobre 5 GB [Neon, sf].

A.1.4. Retrospectiva

Burndown del Sprint 1

La Figura A.1 muestra la evolución de PBIs pendientes del alcance de la entrega (16 PBIs: P1–P14, P18, P19) según la tabla «Evolución del sprint» de `docs/gestion/README.md`. El escalón «+P19» refleja el cambio de alcance al añadir el despliegue en Cloud Run.

Seguimiento visual en DoneTonic

La Figura A.2 muestra las gráficas Sprint Burnup y Sprint Burndown de DoneTonic (84% completado, 3 PBIs pendientes del Hito 4), tal como se captura en el panel de seguimiento del proyecto.

Cambio de alcance

Durante el sprint, el profesor repartió créditos de Google Cloud para educación. Para aprovecharlos se añadió el PBI **P19** (despliegue continuo en Google Cloud Run) con sus tareas T29 y T30. Render se mantuvo como despliegue alternativo. Este cambio se refleja en

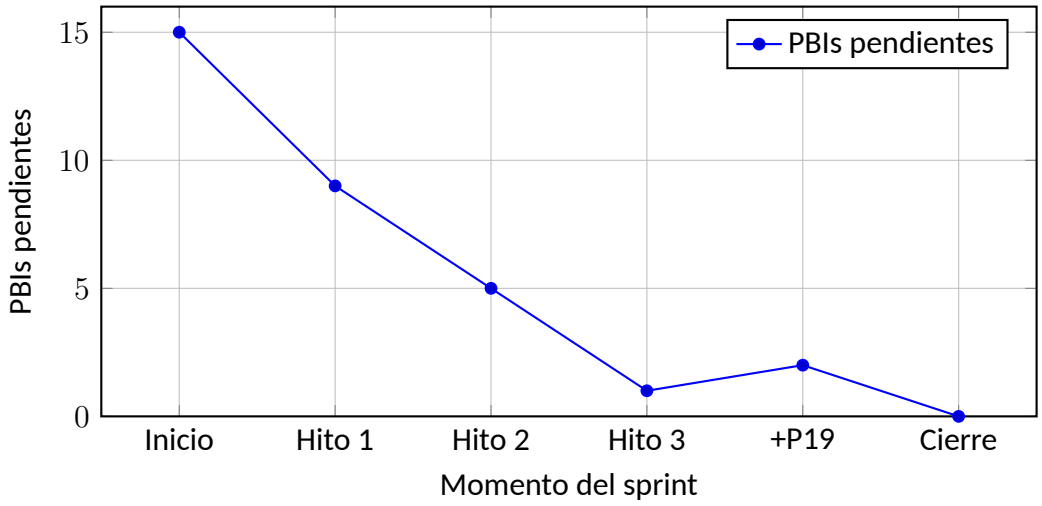


Figura A.1: Burndown del Sprint 1: evolución de PBIs pendientes del alcance de la entrega.

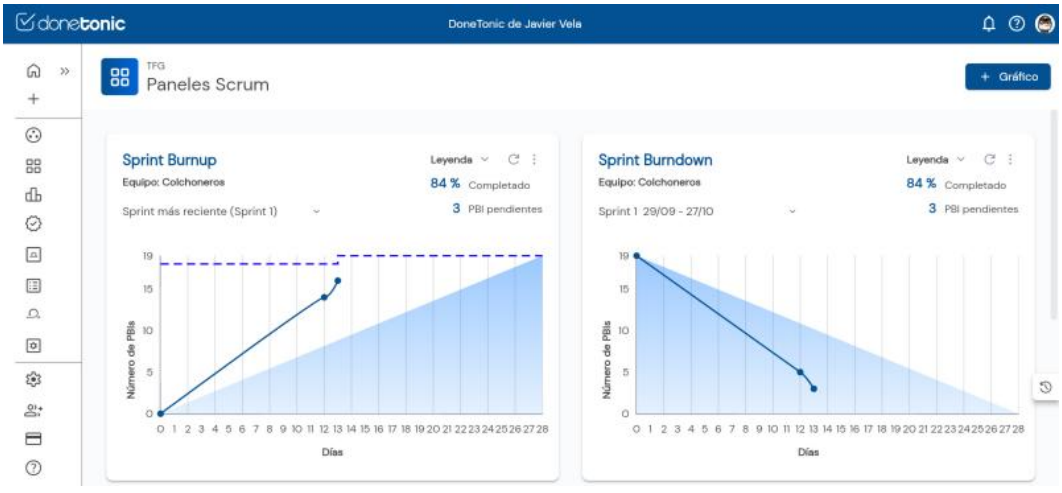


Figura A.2: Sprint Burnup y Sprint Burndown en DoneTonic (84% completado, 3 PBIs pendientes).

el burndown como un incremento temporal de PBIs pendientes (de 1 a 2) antes del cierre final.

Lo aprendido

- La disciplina de GitHub Flow (rama por tarea, PR, CI obligatorio) garantiza que no se integra código roto, pero exige sincronización continua de ramas locales para evitar conflictos de avance rápido.
- El patrón repositorio con inyección de dependencias permitió sustituir el almacenamiento en memoria por PostgreSQL sin modificar una sola línea de la capa de lógica, respaldado por pruebas de contrato idénticas.
- La auditoría manual frase a frase frente al enunciado oficial demostró ser indispensable, localizando discrepancias de acceso que habían pasado inadvertidas en la formulación de los PBIs.
- La concentración cronológica de commits en una jornada (12/10/2026) preserva la trazabilidad estructural del proceso, si bien se reconoce la conveniencia pedagógica de una cadencia temporal más distribuida.
- Todo secreto compartido involuntariamente en canales de comunicación debe ser rotado como buena práctica de seguridad.

B. Material Adicional

B.1. Enlaces del proyecto

- **Repositorio principal:**
  `https://github.com/VelaElDios/la-grada-colchonera-saas`
- **Aplicación principal (Google Cloud Run):**
  `https://la-grada-colchonera-1081564062572.europe-southwest1.run.app`
- **Aplicación alternativa (Render):**
  `https://la-grada-colchonera.onrender.com`
- **Releases publicadas (Hitos 1–3 y sprint-1):**
  `https://github.com/VelaElDios/la-grada-colchonera-saas/releases`
- **Documentación técnica del repositorio:**
  - `README.md`: visión general, tecnologías, ejecución local, pruebas, variables, API, despliegue y acceso administrador.
  - `CONTRIBUTING.md`: flujo de trabajo con ramas, commits y PR.
  - `docs/revision-entrega.md`: requisitos del enunciado y su evidencia.
  - `docs/gestion/README.md`: backlog, trazabilidad tarea → rama → PR, burndown y capturas de DoneTonic.
  - `memoria/README.md`: instrucciones para compilar la memoria.

B.2. Credenciales de administrador para la revisión

Las credenciales del primer administrador (valores de `ADMIN_EMAIL` y `ADMIN_PASSWORD`) se envían junto con la entrega, tal como se indica en el `README.md` (sección «Acceso como administrador para la revisión»). No están en el repositorio ni en esta memoria.

C. Uso Responsable de la Inteligencia Artificial

De acuerdo con las directrices académicas sobre honestidad y rigor en la autoría de trabajos universitarios, este anexo documenta con absoluta transparencia el uso de asistentes basados en modelos de inteligencia artificial a lo largo de las distintas etapas del proyecto.

C.1. Uso de la inteligencia artificial en el desarrollo del software

Durante la ejecución técnica del Sprint 1 se utilizó el modelo Claude (Cowork) como asistente de desarrollo para agilizar las tareas de implementación. Su papel abarcó la redacción del código fuente en JavaScript, la preparación de ficheros de configuración, la elaboración de la documentación técnica y la generación de los mensajes de confirmación de cambios. Dichos commits quedaron registrados con la autoría formal del alumno y el correspondiente metadato de colaboración (`Co-Authored-By`). Asimismo, el asistente gestionó en el navegador la apertura y fusión de los pull requests y la actualización de los estados en el tablero ágil de DoneTonic.

No obstante, el control del entorno productivo y la gobernanza del proyecto recayeron en todo momento en el alumno:

- La creación, administración y custodia de credenciales en las plataformas de infraestructura (Neon para PostgreSQL, Render y Google Cloud Run) fueron realizadas personalmente por el alumno.
- El aseguramiento de la calidad técnica no se delegó en la generación sintética de la IA, sino en una suite de 80 pruebas automatizadas ejecutadas obligatoriamente en el pipeline de integración continua ante cada cambio propuesto.
- El alumno llevó a cabo una auditoría manual frente a las especificaciones oficiales (`docs/revision-entrega.md`), detectando y corrigiendo oportunamente una discrepancia de permisos en la comprobación de usuarios activos (PR #30).

C.2. Uso de la inteligencia artificial en la redacción de la memoria

Para la composición de esta memoria técnica se empleó un flujo automatizado denominado «Cerebro Universal», orquestado sobre GitHub Actions y sustentado en modelos de lenguaje gratuitos. En dicho sistema, una IA constructora generó las secciones del documento y una IA revisora analizó la consistencia de cada avance.

La fidelidad de la información redactada se garantizó mediante estrictas salvaguardas metodológicas:

- Los modelos operaron exclusivamente sobre la información factual contenida en la guía del proyecto y los documentos reales del repositorio, quedando descartada cualquier inferencia externa no contrastada.
- Un proceso de verificación algorítmica independiente comprobó que el documento compilase sin fallos, sin marcadores de relleno de plantilla, con todas las citas bibliográficas resueltas y certificando que ningún dato cuantitativo superior a 20 careciese de respaldo en las fuentes.
- Claude intervino en la preparación de la estructura inicial de la plantilla en LuaLaTeX, las fuentes de datos y las pruebas de comprobación sintáctica.

El alumno asume la autoría y responsabilidad íntegra sobre el contenido final expuesto en la presente memoria, habiendo revisado minuciosamente el texto para certificar su rigor técnico y su estricta correspondencia con el trabajo desarrollado.

Referencia bibliográfica

[Express, sf] Express (s.f.). Express, node.js web application framework. https://expressjs.com/.

[GitHub, sfa] GitHub (s.f.a). Github actions documentation. https://docs.github.com/en/actions.

[GitHub, sfb] GitHub (s.f.b). Github flow. https://docs.github.com/en/get-started/using-github/github-flow.

[Google Cloud, sf] Google Cloud (s.f.). Cloud run documentation. https://cloud.google.com/run/docs.

[Neon, sf] Neon (s.f.). What are the limits and quotas for neon's free plan? https://neon.com/faqs/free-plan-limits-and-quotas.

[OpenJS Foundation, sf] OpenJS Foundation (s.f.). Node.js. https://nodejs.org/.

[Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide. https://scrumguides.org/scrum-guide.html.

[The PostgreSQL Global Development Group, sf] The PostgreSQL Global Development Group (s.f.). Postgresql. https://www.postgresql.org/.

[Universidad de Castilla-La Mancha, 2026] Universidad de Castilla-La Mancha (2026). Especificación del proyecto. procesos de ingeniería del software, curso 2026-2027 (sprint 1).